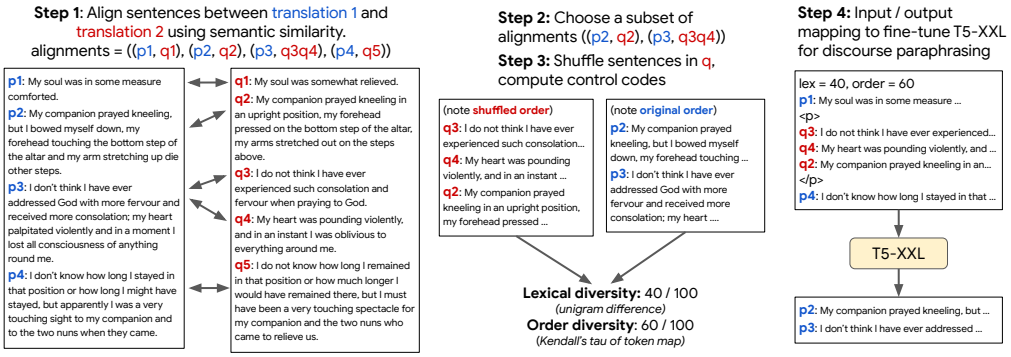

Figure 2: The method used to train DIPPER on English translations of the French novel *The Nun*. We first align sentences between the two translations to create parallel data. Next, a subset of the alignments are chosen; in this example, we use (p_2, q_2) and (p_3, q_3q_4). We shuffle sentences, compute control codes, and fine-tune a T5-XXL LM to generate p_2p_3 given $q_3q_4q_2$ and the context p_1 and p_4.

denote sentences of q. Note that M may not be equal N when two translators disagree on when to merge and split sentences. We perform the following steps (overview in Figure 2):

1. **Align sentences** of p to sentences of q by using the semantic similarity scores from the paraphrase similarity metric in Wieting et al. [2019] to run the sequence alignment algorithm designed by Needleman and Wunsch [1970] which uses dynamic programming (metric details in Section 4.1).
2. **Choose a subset of sentences** $p_i ... p_j$ from the first paragraph. Let $q_{i'} ... q_{j'}$ be the corresponding alignment in the second paragraph. In Figure 2, $i = 2, j = 3, i' = 2, j' = 4$.
3. **Re-order** the sentences $q_{i'} ... q_{j'}$ randomly, and compute the **diversity control codes** between $p_i ... p_j$ and shuffle$(q_{i'} ... q_{j'})$. We shuffle the sentences to allow for the model to learn content re-ordering. We compute lexical diversity (L) using unigram token overlap (F1 score), and the order diversity (O) using the Kendall-Tau correlation of tokens of overlapping words between $p_i ... p_j$ and shuffle$(q_{i'} ... q_{j'})$, also used in Krishna et al. [2020]. These scores are normalized to values $\{0, 20, 40, 60, 80, 100\}$, where $L = 20$ roughly corresponds to a 20% lexical modification.
4. **Map** the shuffled $q_{i'} ... q_{j'}$ to $p_i ... p_j$, leveraging context from the rest of p and control codes using string concatenation. Let input $=$ shuffle$(q_{i'} ... q_{j'})$. We map,

$$\text{lexical} = L, \text{order} = O \oplus p_1 ... p_{i-1} \oplus \texttt{<p>} \text{ input } \texttt{</p>} \oplus p_{j+1} ... p_N \longrightarrow p_i ... p_j$$

where $\oplus$ is string concatenation. During inference, we can paraphrase any sequence of sentences by marking it with <p> tags, assigning the control codes (L, O) the desired diversity values.

Our final dataset contains 6.3M paraphrase pairs. We **fine-tune** a sequence-to-sequence Transformer [Vaswani et al., 2017] on this data, initialized with the pretrained 11B parameter T5-XXL checkpoint [Raffel et al., 2020]. See Appendix F.1 for details.

4 Experiments attacking detection algorithms with DIPPER

In this section, we describe our experimental setup in Section 4.1-4.2 and present our results in Section 4.3. Overall, we find that DIPPER evades all detectors across three LLMs (including GPT3.5).

4.1 Evaluation metrics

Detection accuracy: Our first metric measures how often the input text is correctly detected as AI-generated. Since detection rates are heavily dependent on the chosen detection threshold, the AUC-ROC metric is commonly used to measure detector performance [Mitchell et al., 2023], which considers the range of all possible thresholds. However, in this application, it is critical that the *false positive rate* (FPR) is low; in other words, human-written text must almost never be classified as machine-generated [OpenAI, 2023a, Kirchenbauer et al., 2023a]. Hence, we fix the FPR to 1% for all detection algorithms (although even 1% is likely too high in practice), and adjust the detection threshold accordingly while reporting detection accuracies. Additionally, we also plot ROC curves focusing on the 0-1% FPR region. Overall, we expect detection rates to plummet on paraphrased text.

Table 1: Performance of detection algorithms (at 1% FPR) before and after DIPPER paraphrasing on **open-ended generation** using Wikipedia prompts (300 generated tokens). As the diversity (L,O) increases, detection rates decrease across algorithms, with nearly perfect semantic similarity (Sim). *GPT3.5 DetectGPT scores computed using 200 samples at 20% FPR as it scores 0% at a 1% FPR.

Metric →	Sim ↑	Detection Accuracy ↓				
Detector →		Watermarks	DetectGPT	OpenAI	GPTZero	RankGen
GPT2-1.5B	-	100.0	70.3	21.6	13.9	**13.5**
+ DIPPER 20L	99.2	97.1	28.7	19.2	9.1	15.8
+ DIPPER 40L	98.4	85.8	15.4	17.8	7.3	18.0
+ DIPPER 60L	96.9	68.9	8.7	**13.3**	7.1	19.8
+ DIPPER 60L, 60O	94.3	**57.2**	**4.6**	14.8	**1.2**	28.5
OPT-13B	-	99.9	14.3	11.3	8.7	**3.2**
+ DIPPER 20L	99.1	96.2	3.3	11.8	5.4	5.2
+ DIPPER 40L	98.6	84.8	1.2	11.6	3.8	6.6
+ DIPPER 60L	97.1	63.7	0.8	**9.1**	6.3	9.3
+ DIPPER 60L, 60O	94.6	**52.8**	**0.3**	10.0	**1.0**	13.5
GPT-3.5-175B, davinci-003	-	-	26.5*	30.0	7.1	**1.2**
+ DIPPER 20L	97.6	-	12.5*	20.6	4.3	1.7
+ DIPPER 40L	96.7	-	8.0*	22.4	4.8	2.0
+ DIPPER 60L	94.2	-	7.0*	**15.6**	6.1	3.9
+ DIPPER 60L, 60O	88.4	-	**4.5***	**15.6**	**1.8**	7.3
Human Text	-	1.0	1.0	1.0	1.0	1.0

Semantic similarity (Sim): Detection accuracy is an insufficient evaluation of our attack's success. We also need to measure whether the original and paraphrased generations share approximately the same semantics. We measure semantic similarity using the state-of-the-art semantic similarity model P-SP from Wieting et al. [2022], an embedding averaging model trained on a large corpus of filtered paraphrase data [Wieting and Gimpel, 2018]. P-SP is a well-calibrated metric that performs well on semantic calibration tests as well as plagiarism detection in STS benchmarks [Agirre et al., 2016]. P-SP is also robust against topically similar non-paraphrases. We found that P-SP it scores just 0.09 on random pairs of paragraphs from the same book (topically similar paragraphs but not paraphrases) in the PAR3 dataset [Thai et al., 2022]. In contrast, the average P-SP score of actual human paraphrase pairs in PAR3 is 0.76. We consider semantics to be approximately preserved if the P-SP score is greater than this average human paraphrase score of 0.76.

Besides semantic similarity, we conduct several automatic evaluations, ablation studies, and human evaluations of intrinsic paraphrase quality in Appendix C.

4.2 Models, datasets & detection algorithms

Base language models: We conduct attacks on three language models of varying sizes that belong to different model families. We consider the GPT2-XL model (1.5B parameters) [Radford et al., 2019], the OPT-13B model [Zhang et al., 2022], and the `text-davinci-003` variant from the GPT-3.5 family [Brown et al., 2020], which has 175B parameters and has additionally been instruction tuned using reinforcement learning from human feedback [Ouyang et al., 2022]. For all LMs, we sample generations that are 300 tokens long before passing them through DIPPER for the attack experiments.[5]

Natural language generation tasks: We experiment with two long-form text generation tasks, since most malicious applications (e.g., fake article creation) are associated with long-form outputs. First, we consider *open-ended generation*, where an LM generates a continuation to a two-sentence prompt. To obtain our prompts, we sample 3K contiguous two-sentence chunks from the validation split of WikiText-103 [Merity et al., 2017] and use the next 300 tokens as the "human-written" continuation. Second, we evaluate *long-form question answering* [Fan et al., 2019], in which an LM answers a question with a 300-word answer (dataset details in Appendix F.2). For our main results, the human

[5]For GPT2-XL and OPT-13B, we generate text using nucleus sampling [Holtzman et al., 2020] with $p = 0.9$. For `davinci-003`, we use the default hyperparameters on the API Playground (temperature = 0.7).

reference answers or continuations are only used to adjust detection thresholds of studied methods to maintain a 1% FPR.[6] Note that we are not removing human-written text from our test set. Our metric is equivalent to having a test set with a 50-50 split between machine/human-written text for the same prompts, and observing the FPR=1% point in the ROC curve (also provided in Appendix H).

Detection algorithms: We attack five detectors:[7] (1) watermarking [Kirchenbauer et al., 2023a]; (2) DetectGPT [Mitchell et al., 2023]; (3) GPTZero [Tian, 2023]; (4) OpenAI's text classifier [OpenAI, 2023a];[8] and (5) RankGen-XL-all [Krishna et al., 2022a].[9] We use the default hyperparameters for each detector. We also respect their minimum length specifications, discarding instances where any of the AI-generated text, human-written text, or paraphrased text is shorter than the minimum length.

Paraphrasing AI-generated text: We pass the prompts for each task and AI-generated responses to those prompts through DIPPER. Specifically, we feed the model input of the form,

lexical = L, order = O prompt
 generated-text ,

where L and O represent the paraphraser diversity control codes and the and special tokens mark the boundaries of the text to be paraphrased. We use $L = 20, 40, 60$ and $O = 0, 60$ in our main attack experiments. After paraphrasing, we ensure that the AI-generated sequence, paraphrased sequence, and human-written sequence have an equal number of words by truncating them to the length of the shortest among the three. To ensure higher semantic preservation, we iteratively paraphrase long sequences three sentences at a time, keeping already paraphrased text in the context of the generation. To show the effectiveness of our attack, we only **paraphrase each generation once**, rather than draw multiple samples until it evades detection.[10]

Figure 3: Detector performance (at 1% FPR) on **long-form QA** before/after paraphrasing. As diversity (L,O) increases, detection rates decrease with very high semantic preservation (Sim). WM: Watermark, D.GPT: DetectGPT, O.AI: OpenAI. *GPT3.5 D.GPT uses 100 samples at 20% FPR to show attack success, as it scores 0% at 1% FPR.

Metric →	Sim ↑	Detection Accuracy ↓		
		W.M.	D.GPT	O.AI
GPT2-1.5B	-	100.0	74.9	59.2
+ DIPPER 20L	99.5	98.9	45.7	35.3
+ DIPPER 40L	99.0	90.7	28.0	34.4
+ DIPPER 60L	97.5	71.1	15.8	**31.3**
+ 60L, 60O	96.2	**55.8**	**7.6**	32.7
OPT-13B	-	100.0	29.8	33.5
+ DIPPER 20L	99.6	98.3	15.0	24.5
+ DIPPER 40L	99.4	87.3	6.4	24.1
+ DIPPER 60L	96.5	65.5	3.2	**21.6**
+ 60L, 60O	92.9	**51.4**	**1.5**	**21.6**
GPT-3.5-175B davinci-003	-	-	67.0*	40.5
+ DIPPER 20L	99.9	-	54.0*	43.1
+ DIPPER 40L	99.8	-	36.0*	43.1
+ DIPPER 60L	99.5	-	23.0*	40.1
+ 60L, 60O	98.3	-	**14.0***	**38.1**
Human Text	-	1.0	1.0	1.0

4.3 Attacking AI-generated text detectors

We present our results in Table 1 and Figure 3. Overall we find that:

Paraphrasing significantly lowers detection accuracy while preserving input semantics. Across all LMs, detectors,[11] and tasks, paraphrasing significantly lowers detection accuracy across all diversity control codes. For instance, paraphrasing GPT2-XL open-ended generations reduces watermark detection accuracy from 100% to 57.2%, and DetectGPT accuracy from 70.3% to just 4.6%. Trends are similar even for large LMs like GPT-3.5, for which paraphrasing reduces OpenAI's text classifier accuracy from 30.0% to 15.6%. Additionally, DIPPER preserves semantics effectively,

[6] Alternatively, if a random half of the human-written data was used for threshold adjustment, we find the other half has a FPR of 0.8%-1.2% across random splits, and this deviation will reduce with a bigger dataset.

[7] We consider both model-specific and model-agnostic detectors, as justified in Appendix F.3.

[8] This classifier was taken down in July 2023 by OpenAI due to its low accuracy.

[9] While RankGen was not explicitly optimized for this task, it was trained to identify well-written continuations, so we hypothesize that it could also act as a reasonable AI-generated text detector.

[10] We discuss this attack briefly in Section 4.4.

[11] Except RankGen, which scores paraphrases as AI-generated more often than non-paraphrased text. We attribute this to paraphrases being poorer continuations to the prompt compared to the original (Appendix C), an aspect RankGen bases its score on. However, it has low overall performance since it is not trained for this task.